

Parity Bit in Error Control Mechanism

A parity bit is a simple error detection mechanism used in digital communications and storage. It helps to identify errors in binary data by adding an extra bit to the data. There are two types of parity bits: even parity and odd parity.

How to Calculate a Parity Bit

1. Even Parity:

- Count the number of 1s in the data.
- If the number of 1s is even, the parity bit is 0.
- If the number of 1s is odd, the parity bit is 1.

2. Odd Parity:

- Count the number of 1s in the data.
- If the number of 1s is even, the parity bit is 1.
- If the number of 1s is odd, the parity bit is 0.

Example

Let's take the binary sequence 1101:

- **Even Parity:**
 - Number of 1s: 3 (odd)
 - Parity bit: 1
 - Resulting sequence: 11011
- **Odd Parity:**
 - Number of 1s: 3 (odd)
 - Parity bit: 0
 - Resulting sequence: 11010

Parity bits are appended at the end of the data sequence, but their position can vary depending on the system.

CHECKSUM

Checksum calculation from ASCII value to hexadecimal and further to binary conversion

For example: - To convert the word “COMPUTER NETWORKS” into hexadecimal and then into binary for checksum calculation, follow these steps:

Step 1: Convert “COMPUTER NETWORKS” to Hexadecimal

Each character in “COMPUTER NETWORKS” is converted to its ASCII value, which is then converted to hexadecimal.

- C: 67 (ASCII) -> 43 (Hex)
- O: 79 (ASCII) -> 4F (Hex)
- M: 77 (ASCII) -> 4D (Hex)
- P: 80 (ASCII) -> 50 (Hex)
- U: 85 (ASCII) -> 55 (Hex)
- T: 84 (ASCII) -> 54 (Hex)
- E: 69 (ASCII) -> 45 (Hex)
- R: 82 (ASCII) -> 52 (Hex)
- Space: 32 (ASCII) -> 20 (Hex)
- N: 78 (ASCII) -> 4E (Hex)
- E: 69 (ASCII) -> 45 (Hex)
- T: 84 (ASCII) -> 54 (Hex)
- W: 87 (ASCII) -> 57 (Hex)
- O: 79 (ASCII) -> 4F (Hex)
- R: 82 (ASCII) -> 52 (Hex)
- K: 75 (ASCII) -> 4B (Hex)
- S: 83 (ASCII) -> 53 (Hex)

So, “COMPUTER NETWORKS” in hexadecimal is:

43 4F 4D 50 55 54 45 52 20 4E 45 54 57 4F 52 4B 53

Step 2: Convert Hexadecimal to Binary

Each hexadecimal digit is converted to its 4-bit binary equivalent.

- **43**: 0100 0011
- **4F**: 0100 1111
- **4D**: 0100 1101
- **50**: 0101 0000
- **55**: 0101 0101
- **54**: 0101 0100
- **45**: 0100 0101
- **52**: 0101 0010
- **20**: 0010 0000
- **4E**: 0100 1110
- **45**: 0100 0101
- **54**: 0101 0100
- **57**: 0101 0111
- **4F**: 0100 1111
- **52**: 0101 0010
- **4B**: 0100 1011
- **53**: 0101 0011

So, “COMPUTER NETWORKS” in binary is:

01000011 01001111 01001101 01010000 01010101 01010100 01000101
01010010 00100000 01001110 01000101 01010100 01010111 01001111
01010010 01001011 01010011

Step 3: Calculate Checksum

Checksum calculation is a method used to ensure data integrity during transmission. Here's a step-by-step process for both the sender and receiver sides:

Sender Side

1. **Divide Data into Blocks:**
 - Split the data into equal-sized blocks, typically 8/16 bits each.
2. **Calculate Sum:**
 - Add all the blocks together using one's complement arithmetic. This means if there's a carry bit, it is added back to the sum.
3. **Complement the Sum:**
 - Take the one's complement of the final sum. This is the checksum.
4. **Append Checksum:**
 - Attach the checksum to the end of the data.

Receiver Side

1. **Receive Data and Checksum:**
 - Receive the data along with the appended checksum.
2. **Divide Data into Blocks:**
 - Split the received data (including the checksum) into equal-sized blocks.
3. **Calculate Sum:**
 - Add all the blocks together using one's complement arithmetic.
4. **Complement the Sum:**
 - Take the one's complement of the final sum.

5. Verify Integrity:

- If the result is all zeros, the data is considered error-free. If not, it indicates that the data has been corrupted.

Example:

Let's consider a simple example with 8-bit blocks for illustration:

Sender Side:

- Data: 10101001 00111001
- Sum: $10101001 + 00111001 = 11100010$
- Checksum: 00011101 (one's complement of the sum)
- Transmitted Data: 10101001 00111001 00011101

Receiver Side:

- Received Data: 10101001 00111001 00011101
- Sum: $10101001 + 00111001 + 00011101 = 11111111$
- Complement: 00000000
- Result: All zeros, indicating no error.

Hamming code calculation for both ODD and EVEN parity

The process of calculating Hamming code for both odd and even parity using an example. We'll use the **data bits 1011** for this illustration.

Step-by-Step Process

1. Determine the Number of Parity Bits:

- For 4 data bits, we need 3 parity bits because ($2^3 \geq 4 + 3 + 1$).

2. Position the Parity Bits:

- Place the parity bits at positions that are powers of 2 (1, 2, 4).

3. Fill in the Data Bits:

- Place the data bits in the remaining positions.

Even Parity Calculation

1. Identify Parity Bit Positions:

- Each parity bit covers positions that have a 1 in the corresponding bit of their binary representation.

2. Calculate Parity Bits:

- For even parity, ensure the total number of 1s (including the parity bit) is even.
- Example: For data bits 1011:
- P1 P2 D1 P4 D2 D3 D4
- _ _ 1 _ 0 1 1
- Calculate P1: Covers positions 1, 3, 5, 7 (1, 1, 1) $\rightarrow$ P1 = 1 (to make total even)

- Calculate P2: Covers positions 2, 3, 6, 7 (0, 1, 1) $\rightarrow P2 = 0$ (total already even)
- Calculate P4: Covers positions 4, 5, 6, 7 (1, 0, 1) $\rightarrow P4 = 0$ (total already even)
- Final code with even parity: 1010110

Odd Parity Calculation

1. Identify Parity Bit Positions:

- Similar to even parity, each parity bit covers positions that have a 1 in the corresponding bit of their binary representation.

2. Calculate Parity Bits:

- For odd parity, ensure the total number of 1s (including the parity bit) is odd.
- Example: For data bits 1011:
- P1 P2 D1 P4 D2 D3 D4
- _ _ 1 _ 0 1 1
- Calculate P1: Covers positions 1, 3, 5, 7 (1, 1, 1) $\rightarrow P1 = 0$ (to make total odd)
- Calculate P2: Covers positions 2, 3, 6, 7 (0, 1, 1) $\rightarrow P2 = 1$ (to make total odd)
- Calculate P4: Covers positions 4, 5, 6, 7 (1, 0, 1) $\rightarrow P4 = 1$ (to make total odd)
- Final code with odd parity: 0110111

Summary

- **Even Parity:** Ensures the total number of 1s is even.
- **Odd Parity:** Ensures the total number of 1s is odd.